

Data Structures

Lecture 5: Lists

Soobin Um

Mar. 18, 2026

Recap

Asymptotic Growth Function

- 점근적 성장 함수 $f(n)$
 - $T(n)$ 을 단순화한 결과로, 입력 크기 n 에 따라 실행 시간이 어떤 차수로 성장하는지를 나타냄

$$\text{e.g., } T(n) = 3n^2 + 2n + 5 \rightarrow f(n) = n^2$$

Asymptotic Analysis

- 점근적 시간 복잡도 표기법

- 수행 시간 함수 $T(n)$ 과 단순화된 성장 함수 $f(n)$ 사이의 관계를 상한, 하한, 정확한 차수라는 서로 다른 관점에서 표현하는 방법

Big-O $O(f(n))$

실행 시간이 최대 어느 정도까지 걸리는지 (상한)

" $f(n)$ 보다 더 느리게 동작하지는 않는다"

Big- Ω $\Omega(f(n))$

실행 시간이 최소 어느 정도는 걸린다는 것 (하한)

"아무리 잘 되어도 $f(n)$ 보다 빠르지는 않다"

Big- Θ $\Theta(f(n))$

실행 시간이 $f(n)$ 과 같은 차수로 증가 (상/하한 동시)

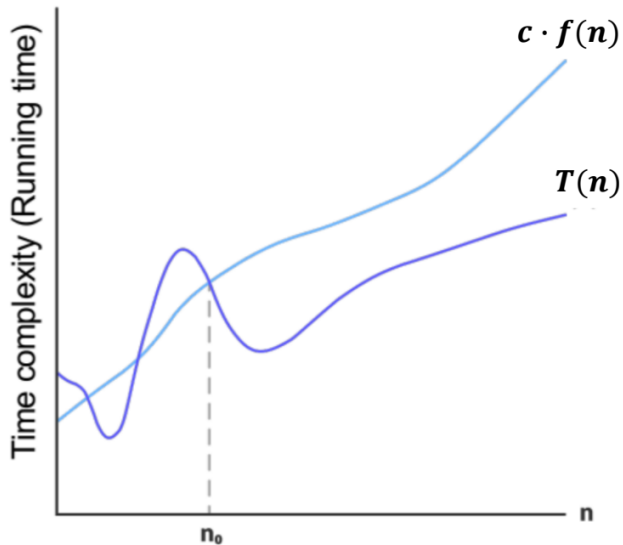
"상/하한이 같은 차수: 실행 시간의 정확한 차수를 전달"

Big-O

• 빅-오 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배 즉, $c \cdot f(n)$ 을 초과하지 않음
- $T(n)$ 은 $f(n)$ 보다 빠르게 증가하지 않음 $\rightarrow$ 점근적 상한 (upper bound)

$T(n) \in O(f(n))$ iff there exist positive constants c and n_0 such that $T(n) \leq c f(n)$ for all $n > n_0$



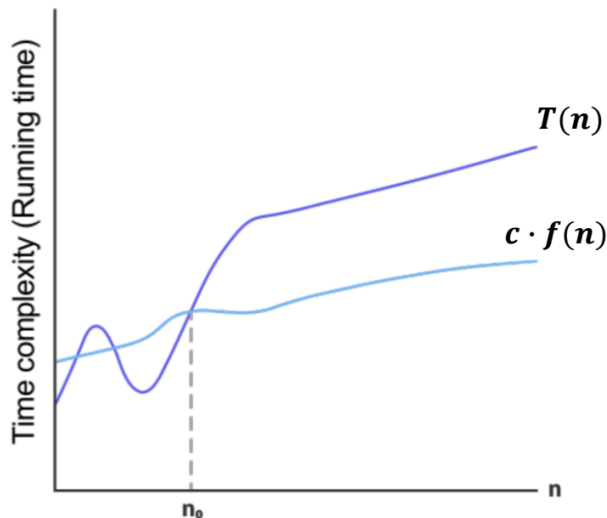
$T(n)$ 은 $f(n)$ 보다 더 빠르게 증가하지 않음
 $\rightarrow T(n)$ 은 $O(f(n))$ 에 속함

Big-Ω

• 빅-오메가 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배, 즉, $c \cdot f(n)$ 이상임
- $T(n)$ 은 $f(n)$ 보다는 빠르게 증가함을 보여주는 점근적 하한 (lower bound) 표현

$T(n) \in \Omega(f(n))$ iff there exist positive constants c and n_0 such that $T(n) \geq c f(n)$ for all $n > n_0$



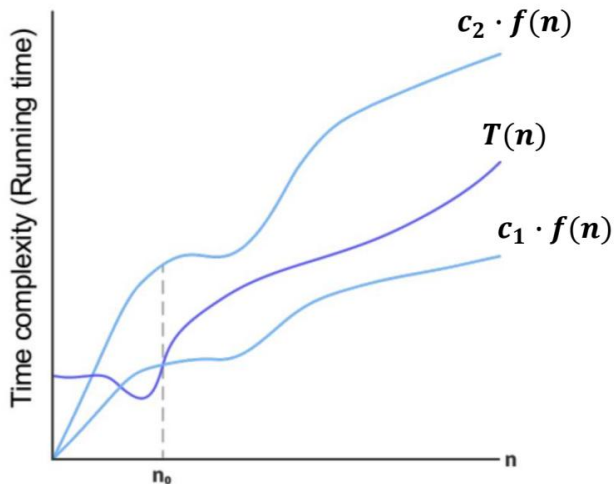
$T(n)$ 은 $f(n)$ 보다 더 빠르게 증가함
→ $T(n)$ 은 $\Omega(f(n))$ 에 속함

Big-Θ

• 빅-세타 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 두 상수배 사이에 위치: $c_1 f(n) \leq T(n) \leq c_2 f(n)$
- $T(n)$ 은 $f(n)$ 과 같은 차수로 증가함을 보여주는 점근적 tight bound 표현

$T(n) \in \Theta(f(n))$ iff there exist positive constants c_1 , c_2 and n_0 such that $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n > n_0$



$T(n)$ 은 $f(n)$ 과 같은 속도로 증가함
→ $T(n)$ 은 $\Theta(f(n))$ 에 속함

Prefix Averages

- 구현 1

- 배열 A 상에서의 각각의 i 마다, $0 \sim i$ 까지의 합산을 구한 뒤 평균

Prefix Averages (Naïve)	# of steps
<pre>01 for (int i=0; i<n; i++) { 02 int sum = 0; 03 for (int j=0; j<i+1; j++) 04 sum += A[j]; 05 PreAve[i] = sum / (i+1); 06 }</pre>	
$T(n) =$	
$T(n) \in O(n^2)$	

Prefix Averages

- 구현 1

- 배열 A 상에서의 각각의 i 마다, $0 \sim i$ 까지의 합산을 구한 뒤 평균

Prefix Averages (Naïve)	# of steps
<pre>01 for (int i=0; i<n; i++) { 02 int sum = 0; 03 for (int j=0; j<i+1; j++) 04 sum += A[j]; 05 PreAve[i] = sum / (i+1); 06 }</pre>	$1+(n+1)+n$
	n
	$\sum_{i=0}^{n-1} 1 + (i+2) + (i+1) = n^2 + 3n$
	$\sum_{i=0}^{n-1} 2(i+1) = n^2 + n$
	$3n$
	-
$T(n) =$	$2n^2+10n+2$
$T(n) \in O(n^2)$	

Prefix Averages

- 구현 2

- 누적합을 이용해 한번의 덧셈 연산만으로 평균 계산 수행

Prefix Averages (Efficient)	# of steps
01 <code>int sum = 0;</code> 02 <code>for (int i=0; i<n; i++) {</code> 03 <code> sum += A[i];</code> 04 <code> PreAve[i] = sum / (i+1);</code> 05 <code>}</code>	
$T(n) =$	
$T(n) \in O(n)$	

Prefix Averages

- 구현 2

- 누적합을 이용해 한번의 덧셈 연산만으로 평균 계산 수행

Prefix Averages (Efficient)	# of steps
01 int sum = 0; 02 for (int i=0; i<n; i++) { 03 sum += A[i]; 04 PreAve[i] = sum / (i+1); 05 }	1
	1+(n+1)+n
	2n
	3n
	-
$T(n) =$	7n+3
$T(n) \in O(n)$	

Lists

Lists (리스트)

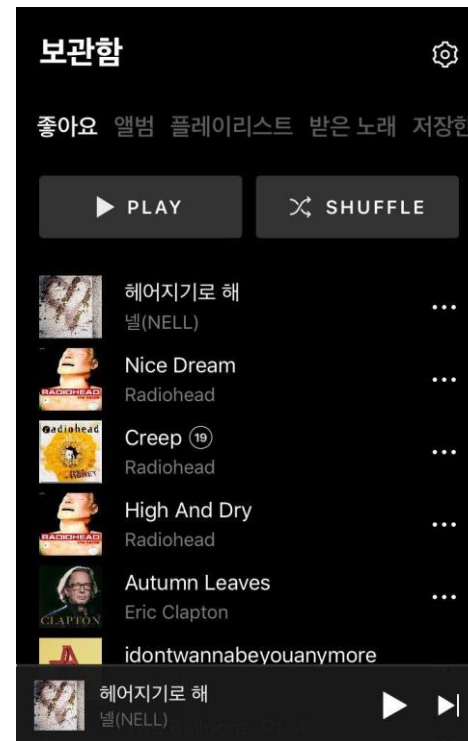
- 순서가 있는 데이터의 나열(`ordered` collection)

Lists (리스트)

- 순서가 있는 데이터의 나열(**ordered** collection)
- 리스트를 구성하는 각 데이터(원소)는 리스트 상에서 각자 자신의 위치(인덱스)를 가짐
 - 인덱스 기반 접근 가능 (예: `list[0]`, `list[1]`)

Lists (리스트)

- 순서가 있는 데이터의 나열(ordered collection)
- 리스트를 구성하는 각 데이터(원소)는 리스트 상에서 각자 자신의 위치(인덱스)를 가짐
 - 인덱스 기반 접근 가능 (예: list[0], list[1])



Key Operations

- **탐색 (Search)**
 - 특정 인덱스의 원소를 찾거나, 특정 값을 가진 원소를 찾는 것
- **갱신 (Update)**
 - 특정 인덱스의 원소를 새로운 값으로 교체

Key Operations

- **탐색 (Search)**
 - 특정 인덱스의 원소를 찾거나, 특정 값을 가진 원소를 찾는 것
- **갱신 (Update)**
 - 특정 인덱스의 원소를 새로운 값으로 교체
- **삽입 (Insert)**
 - 새로운 원소를 원하는 위치에 추가
 - 배열에서의 경우, 뒤의 원소들을 밀어내야 함
- **삭제 (Delete)**
 - 특정 위치의 원소를 제거
 - 배열에서의 경우, 뒤의 원소들을 앞으로 당겨야 함

Key Operations

- **탐색 (Search)**
 - 특정 인덱스의 원소를 찾거나, 특정 값을 가진 원소를 찾는 것
- **갱신 (Update)**
 - 특정 인덱스의 원소를 새로운 값으로 교체
- **삽입 (Insert)**
 - 새로운 원소를 원하는 위치에 추가
 - 배열에서의 경우, 뒤의 원소들을 밀어내야 함
- **삭제 (Delete)**
 - 특정 위치의 원소를 제거
 - 배열에서의 경우, 뒤의 원소들을 앞으로 당겨야 함
- **순회 (Traverse)**
 - 리스트의 모든 원소를 차례대로 읽음
 - 예: 출력, 합계 계산

Applications

- 기본 자료 구조
 - 스택(Stack), 큐(Queue), 데크(Double-ended queue, Deque) 등의 자료 구조는 모두 리스트를 기반으로 구현 가능

Applications

- 기본 자료 구조

- 스택(Stack), 큐(Queue), 데크(Double-ended queue, Deque) 등의 자료 구조는 모두 리스트를 기반으로 구현 가능

- 시스템 SW 분야

- 운영체제: 프로세스 관리(ready list)
- 데이터베이스: 레코드 집합(테이블) 관리



ubuntu



ORACLE®

DATABASE



Recall: ADT (Abstract Data Type)

- 자료 구조와의 비교

- ADT는 개념적 설계도
- 자료 구조는 그 설계도를 바탕으로 만든 실제 구현물

	Abstract Data Type (ADT)	Data Structure
정의	데이터와 연산을 개념적으로 정의한 것	ADT를 실제로 구현한 것
관점	"무엇"을 할 수 있는지	"어떻게" 구현할 것인지
예시	스택 (push, pop 연산 정의 포함)	배열 기반 스택, 연결리스트 기반 스택
특징	구현 방법은 숨김 (encapsulation)	메모리 배치, 알고리즘으로 구체화

List ADT

- 구현 방법과 상관없이 인터페이스 차원에서 “**위치를 가진 원소들의 유한 순서열**”을 정의

```
public interface List<E> {  
    public void clear();  
    public void insert(int pos, E item);  
    public void append(E item);  
    public void update(int pos, E item);  
    public E getValue(int pos);  
    public E remove(int pos);  
    public void printAll();  
}
```

※ 제네릭 타입 E: Element의 약자로, 리스트에 어떤 자료형을 담을지를 **나중에 결정**

List ADT: 연산 동작 예

- List L = <12, 32, 15>
- L.insert(1, 99) $\Rightarrow$ <12, 99, 32, 15>
- L.remove(3) $\Rightarrow$ <12, 99, 32>
- L.update(0, 61) $\Rightarrow$ <61, 99, 32>
- L.length() $\Rightarrow$ 3
- L.clear() $\Rightarrow$ <>

```
public interface List<E> {  
    public void clear();  
    public void insert(int pos, E item);  
    public void append(E item);  
    public void update(int pos, E item);  
    public E getValue(int pos);  
    public E remove(int pos);  
    public void printAll();  
}
```

Question: ADT → Data Structure

“리스트를 실제로 어떻게 구현할 수 있을까?”
(= 리스트 상에서의 원소들을 어떻게 줄 세울 수 있을까?)

Question: ADT → Data Structure

“리스트를 실제로 어떻게 구현할 수 있을까?”
(= 리스트 상에서의 원소들을 어떻게 줄 세울 수 있을까?)

배열(array) 기반 구현

: 연속된 빈칸에 데이터를 차례대로 저장

Question: ADT → Data Structure

“리스트를 실제로 어떻게 구현할 수 있을까?”
(= 리스트 상에서의 원소들을 어떻게 줄 세울 수 있을까?)

배열(array) 기반 구현

: 연속된 빈칸에 데이터를 차례대로 저장

연결 리스트(linked list) 기반 구현

: 각 칸마다 옆 칸을 가리키는 화살표를 붙여서 연결

배열 기반 리스트 구현

- 배열과 사이즈(size) 변수를 통해 리스트 구현
 - 배열: 실제 원소를 저장하는 고정 크기 공간
 - 사이즈 변수: 현재 배열에 저장된 원소 개수 관리

배열 기반 리스트 구현

- 배열과 사이즈(size) 변수를 통해 리스트 구현
 - 배열: 실제 원소를 저장하는 고정 크기 공간
 - 사이즈 변수: 현재 배열에 저장된 원소 개수 관리

```
class ArrayListDemo {  
    private int[] arr();  
    private int size;  
    public ArrayListDemo(int capacity) {  
        arr = new int[capacity];  
        size = 0;  
    }  
}
```

배열 기반 리스트 구현

- 배열과 사이즈(size) 변수를 통해 리스트 구현
 - 배열: 실제 원소를 저장하는 고정 크기 공간
 - 사이즈 변수: 현재 배열에 저장된 원소 개수 관리

```
class ArrayListDemo {  
    private int[] arr();  
    private int size;  
    public ArrayListDemo(int capacity) {  
        arr = new int[capacity];  
        size = 0;  
    }  
}
```

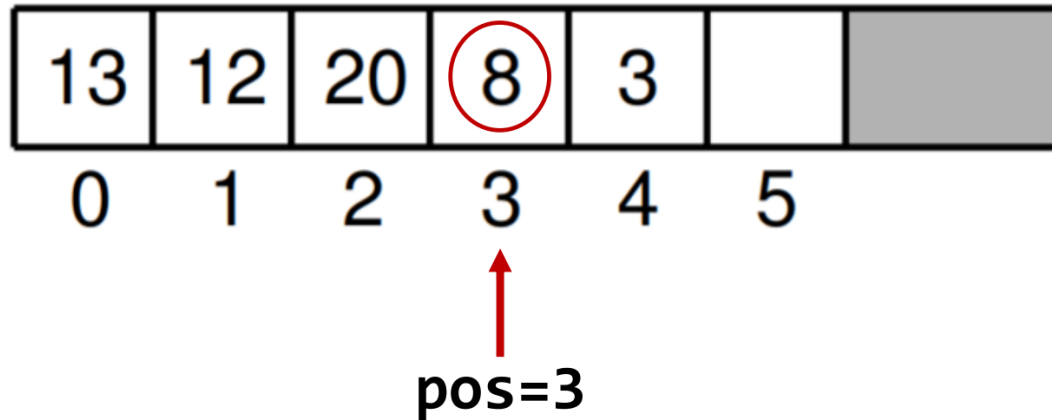
- size 변수의 필요성
 - 배열 상에 실제 저장된 원소 개수 추적
 - 삽입/삭제 연산 관리
 - 순회 시 경계 제어(예: for (int i=0; i<size; i++))

탐색 Search

- 인덱스 접근을 통해 특정 위치의 원소를 빠르게 탐색 가능($\in O(1)$)
 - 단, 특정 값 찾기를 위해선, 선형 탐색이 필요

탐색 Search

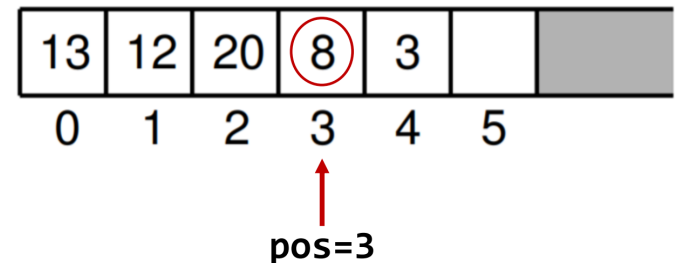
- 인덱스 접근을 통해 특정 위치의 원소를 빠르게 탐색 가능($\in O(1)$)
 - 단, 특정 값 찾기를 위해선, 선형 탐색이 필요



탐색 Search

- 인덱스 접근을 통해 특정 위치의 원소를 빠르게 탐색 가능($\in O(1)$)
 - 단, 특정 값 찾기를 위해선, 선형 탐색이 필요

```
public int getValue(int pos) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    return arr[pos];  
}
```



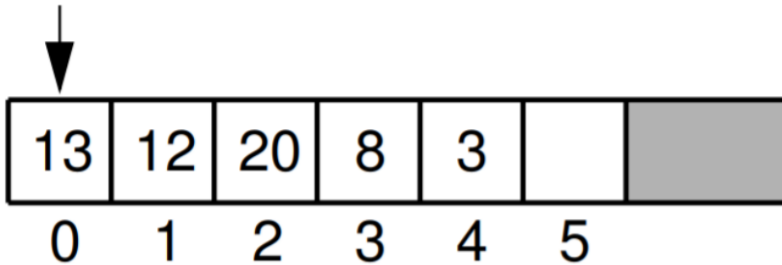
삽입 Insert

- 특정 위치에 새로운 값을 넣기 위해, 뒤에 있는 원소들을 한 칸씩 뒤로 이동($\in O(n)$)

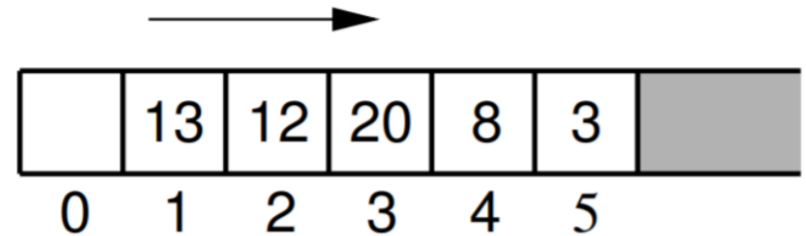
삽입 Insert

- 특정 위치에 새로운 값을 넣기 위해, 뒤에 있는 원소들을 한 칸씩 뒤로 이동($\in O(n)$)

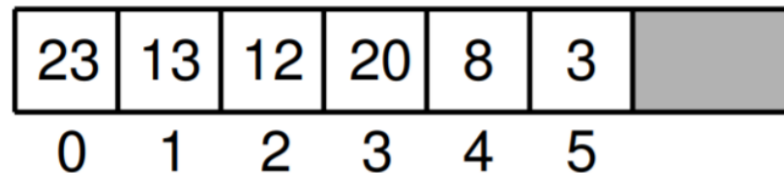
Insert 23:



(a)



(b)



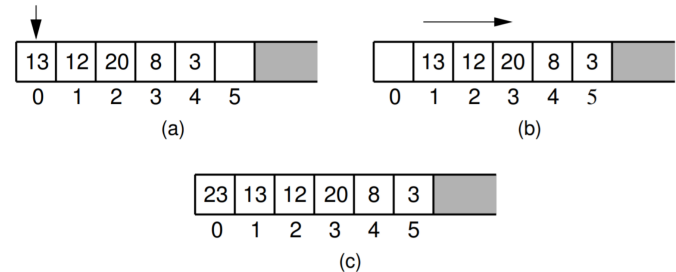
(c)

삽입 Insert

- 특정 위치에 새로운 값을 넣기 위해, 뒤에 있는 원소들을 한 칸씩 뒤로 이동($\in O(n)$)

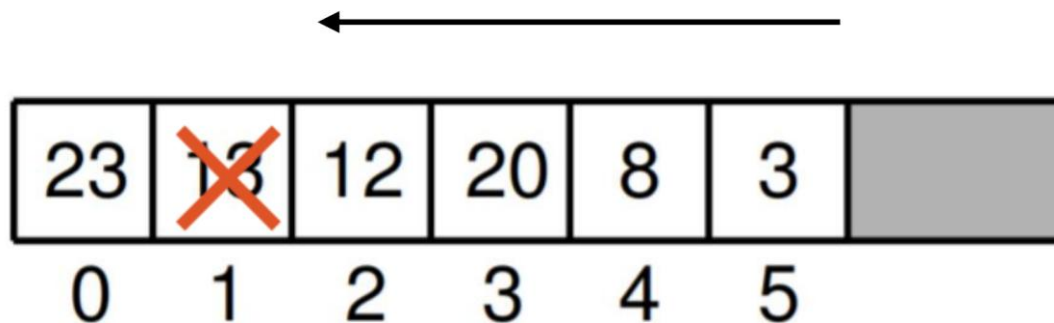
```
public void insert(int pos, int item) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    for (int i = size; i > pos; i--)  
        arr[i] = arr[i-1];  
  
    arr[pos] = item;  
    size++;  
}
```

Insert 23:



삭제 Delete

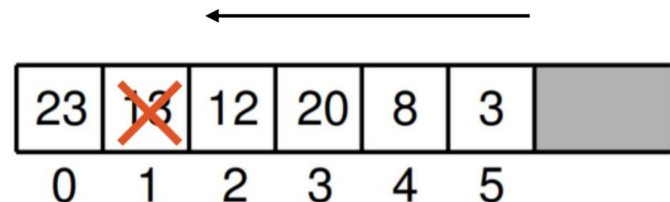
- 특정 위치의 원소를 지우기 위해, 뒤에 있는 원소들을 한 칸씩 앞으로 이동($\in O(n)$)



삭제 Delete

- 특정 위치의 원소를 지우기 위해, 뒤에 있는 원소들을 한 칸씩 앞으로 이동($\in O(n)$)

```
public int remove(int pos) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    int removed = arr[pos];  
    for (int i = pos; i < size - 1; i++)  
        arr[i] = arr[i + 1];  
  
    size--;  
    return removed;  
}
```



순회 Traverse

- 리스트의 모든 원소를 차례대로 출력($\in O(n)$)

```
public void printAll() {  
    for (int i=0; i<size; i++)  
        System.out.println(arr[i] + " ");  
  
    System.out.println();  
}
```

Pros and Cons

- **장점**

- 특정 위치의 데이터로의 매우 빠른 접근 가능($\in O(1)$)
- 구현이 단순하고 직관적
- 메모리에 연속 저장되므로 캐시 친화적

Pros and Cons

- **장점**

- 특정 위치의 데이터로의 매우 빠른 접근 가능($\in O(1)$)
- 구현이 단순하고 직관적
- 메모리에 연속 저장되므로 캐시 친화적

- **단점**

- 삽입/삭제 연산 시 많은 원소 이동 필요 ($\in O(n)$)
- 크기가 고정되어 있음
 - 넣으려는 데이터가 더 많을 경우, 에러 발생
 - 넣으려는 데이터가 너무 적을 경우, 메모리 낭비

Look ahead

연결 리스트(Linked Lists)